

# Integración con `cv-filter-api` -- guía para la plataforma web

---

Documento para el equipo que consume el motor de screening desde el backend de la plataforma.

*\*\*Nota sobre el storage\*\*: hoy el motor usa Google Cloud Storage (^gs://). El contrato de integración --prefijos, nombres de archivo, formato de los JSON-- está pensado para ser portable a S3 (^s3://) u otro object storage compatible con minimal changes en el motor. La plataforma debe poder switchear el cliente del bucket (SDK de GCS vs. SDK de S3) sin que cambie el resto del flujo. En este documento se usa ^<storage>:// cuando el ejemplo aplica indistinto.*

---

## Flujo end-to-end

```
1. Plataforma genera job_id (string único)
2. Plataforma sube los CVs al bucket -> <storage>://$BUCKET/jobs/{job_id}/cvs/
3. Plataforma -> POST /jobs (dispara el procesamiento)
4. Plataforma -> GET /jobs/{job_id} (poll de status, cada N segundos)
5. Cuando completed + failed == total -> leer resultados de
    <storage>://$BUCKET/jobs/{job_id}/results/
```

---

## 1) Qué tiene que subir al bucket ANTES de llamar al API

**Prefijo del bucket** (uno por job): `<storage>://$BUCKET/jobs/{job_id}/cvs/`

- Un objeto por CV. El nombre del archivo se conserva tal cual (se usa como key del resultado).
- Formatos soportados:
  - Texto: `.pdf`, `.docx`, `.txt`, `.md`
  - Imagen: `.png`, `.jpg`, `.jpeg`, `.bmp`, `.tiff`, `.webp`
- Cualquier extensión fuera de esa lista será ignorada (no devuelve error, simplemente no se procesa).

El `job_id` debe ser único globalmente (el bucket es compartido, no hay multi-tenant en el motor).

Sugerencia: usar UUID o prefijar con el id del cliente/tenant interno.

---

## 2) `POST /jobs` -- disparar el procesamiento

Llamar **después** de haber terminado de subir todos los CVs a `jobs/{job_id}/cvs/`.

**Request body (JSON):**

```
{
  "job_id": "abc-123",
  "job_description": "Texto completo de la JD...",
  "model_tier": "cheap"
}
```

**Notas:**

- `model_tier` (opcional, default `"cheap"`): `"cheap"` | `"balanced"` | `"accurate"`.
- El API valida que existan CVs en el bucket. Si no hay -> 400.

#### Response 202 (Accepted):

```
{
  "job_id": "abc-123",
  "total_cvs": 47,
  "status_url": "/jobs/abc-123"
}
```

#### Códigos de error:

- **400**: `model_tier` inválido, o no hay CVs en el prefijo del bucket.
- **500**: bucket no configurado en el servidor.

**Auth:** el endpoint corre con `--no-allow-unauthenticated`. La plataforma debe llamar con un token de identidad de una SA/IAM principal con permiso para invocar el Service (en GCP: `roles/run.invoker`; en AWS equivalente, según cómo quede deployado).

---

### 3) ``GET /jobs/{job_id}` -- status`

#### Response 200:

```
{
  "job_id": "abc-123",
  "total": 47,
  "completed": 30,
  "failed": 2,
  "pending": 15,
  "cost_usd": 0.0142,
  "results_uri": "gs://mi-bucket/jobs/abc-123/results/"
}
```

- `total = completed + failed + pending`.
- El job terminó cuando ``pending == 0`` (i.e. `completed + failed == total`).
- `cost_usd` es acumulado y crece a medida que se completan CVs.
- El status se deriva listando el bucket -- no hay un campo "done" booleano, se infiere.

**Polling sugerido:** cada 5-15s. No hay webhooks por ahora.

---

### 4) Resultados -- qué se escribe en el bucket

**Éxito:** ``<storage>://$BUCKET/jobs/{job_id}/results/{nombre_cv_original}.json``

Ejemplo (`juan_perez.pdf.json`):

```
{
  "output_llm": {
    "score_llm": 78,
    "datos_contacto": {
      "nombre": "Juan Pérez",
      "email": "juan@example.com",
      "telefono": "+54 11 5555-5555",
    }
  }
}
```

```

    "ubicacion": "Buenos Aires, Argentina",
    "links": ["https://linkedin.com/in/juanperez"],
    "edad": 32
  }
},
"nombre_archivo_cv": "juan_perez.pdf",
"score_final": 78,
"model": "gemini/gemini-2.5-flash-lite",
"model_tier": "cheap",
"input_tokens": 1842,
"output_tokens": 312,
"cost_usd": 0.000128,
"latency_s": 2.41,
"processed_at": "2026-05-20T15:23:11.482Z"
}

```

Campos relevantes para la UI:

- **score\_final** (0-100) -> ranking de candidatos.
- **output\_llm.datos\_contacto** -> datos extraídos del CV.
- **nombre\_archivo\_cv** -> para mapear de vuelta al archivo original.
- **cost\_usd**, **latency\_s**, **model** -> telemetría/auditoría.

**Error: `<storage>://\$BUCKET/jobs/{job\_id}/errors/{nombre\_cv\_original}.json`**

```

{
  "cv_name": "cv_corrupto.pdf",
  "error": "PDF malformed: ...",
  "traceback": "Traceback (most recent call last):\n  ...",
  "failed_at": "2026-05-20T15:24:02.117Z"
}

```

Errores per-CV no tumban el job -- el job termina OK incluso si algunos CVs fallaron.

## 5) Layout final del bucket por job

```

<storage>://$BUCKET/jobs/{job_id}/
??? cvs/                                <- subidos por la plataforma ANTES del POST /jobs
?   ??? juan_perez.pdf
?   ??? maria_lopez.docx
?   ??? foto_cv.jpg
??? job.json                            <- metadata del job
??? results/                            <- uno por CV exitoso
?   ??? juan_perez.pdf.json
?   ??? maria_lopez.docx.json
??? errors/                             <- uno por CV fallado
    ??? foto_cv.jpg.json

```

## 6) Lo que la plataforma necesita acordar con infra

- **Identidad/credenciales** con permisos para:

- Escribir y leer el prefijo `jobs/{job_id}/` del bucket.
  - Invocar el Service `cv-filter-api`.
  - **URL del Service.**
  - **Nombre del bucket** y provider (GCS o S3 cuando aplique).
  - **Convención de `job\_id`** que evite colisiones entre tenants/clientes (el motor no aísla nada por sí mismo).
- 

## 7) Edge cases a tener en cuenta

- **Reintentos del POST:** si la plataforma reintentea POST con el mismo `job_id` y parte de los CVs ya están procesados, el motor es idempotente -- saltea los que ya tienen `results/*.json` y sólo procesa los que falten. Útil para "resume after crash".
- **Eliminación de jobs antiguos:** el motor no limpia nada. La plataforma o un job de housekeeping aparte debe borrar prefijos viejos.
- **Tamaño de CV:** el texto se trunca a 30k chars antes del LLM (suficiente para la gran mayoría de los CVs reales).
- **Sin webhook de finalización:** hoy es sólo polling. Si se necesita, hay que agregarlo del lado del motor.
- **Portabilidad GCS -> S3:** el contrato del bucket (prefijos, nombres, formato JSON) es agnóstico al provider; la plataforma debería abstraer el cliente del bucket de manera que cambiar de GCS a S3 sea sólo cambiar el SDK + las credenciales.